

Set 2

Below you can find the instructions for each exercise in **Set 2**.

Similar to Set 1, you can create a workspace for your **Set2** folder in **VS Code** by selecting **Open Folder...** and choosing your **Set2** folder.

This allows you to access and manage all your exercise files in one place.

Open the **.c** file you want to run using **VS Code** or any other text editor you prefer (for example, Notepad on Windows).

You can edit your C file directly in any of these editors.

When you finish writing your code, you need to **compile it**.

To do this, open **w64devkit** and navigate to the **exercise folder** you are working on within your **Set2** directory.

For detailed steps on how to navigate to the folder and compile your program, refer to the *"Getting started with LEU483"* guide on Canvas.

You do not need to complete all exercises at once. Some can be saved for exam practice.

However, it is recommended that you complete **Exercises 1 to 8** before starting **Assignment 1a**.

Note: Make sure to save your work on the **Z-disc**, not on the C-disc.

1 Sum to

Open: `ex_sum_to/main.c`

Samples: `sample_for_stmt`

Task:

Write a program that calculates the sum from 1 to an integer provided through user input. The integer should be included in the sum. You should use a for-loop.

Example output:

```
Program to sum 1-n      (first run)
Input n : 1
Sum 1 to 1 = 1

Program to sum 1-n      (second run)
Input n : 6
Sum 1 to 6 = 21         (1+2+3+4+5+6)
```

2 Factorial

Open: `ex_factorial/main.c`

This is a program that calculates the factorial of an integer provided through user input. See <https://en.wikipedia.org/wiki/Factorial> for an explanation of the factorial. Note that the value of the

factorial can increase very quickly, so think about what type to use for the variable storing the result.

Task:

- Complete the program such that it calculates the factorial of the number provided by the user.
- Use `sizeof` to print the size of the variable `factorial`.
- Change the type of `factorial` to `int`, what happens?

Example output (for correct type of `factorial`):

```
Enter an integer (less than 20): 3      (first run)
Factorial = 6

Enter an integer (less than 20): 6      (second run)
Factorial = 720

Enter an integer (less than 20): 15     (last run)
Factorial = 1307674368000
```

3 Quadratic function

Open: `ex_quad/main.c`

Samples: `sample_math`

Task:

Write a program that solves the quadratic formula $ax^2 + bx + c = 0$. Use functions from `math.h`. For help with the quadratic formula see:

- https://en.wikipedia.org/wiki/Quadratic_formula
- <https://www.mathsisfun.com/quadratic-equation-solver.html>

Note: When you use functions from `math.h` (for example, `sqrt`, `pow`, or `sin`), you must link the math library when compiling your program. Do this by adding `-lm` at the end of your compile command.

For example:

```
$ gcc main.c -o quad_solver.exe -lm
```

If you forget to add `-lm`, you may see an error such as `undefined reference to 'sqrt'`.

Example output:

```
Program to solve a quadratic equation:  $ax^2 + bx + c = 0$       (first run)
```

```
Please input a > 1
```

```
Please input b > 2
```

```
Please input c > 3
```

```
There are two complex conjugate roots. Can't solve will give nan
```

```
x1 = -nan, x2 = -nan
```

```
Program to solve a quadratic equation:  $ax^2 + bx + c = 0$       (second run)
```

```
Please input a > 1
```

```
Please input b > 2
```

```
Please input c > 1
```

```
There are two equal real roots x1 = -1.000000, x2 = -1.000000
```

```
Program to solve a quadratic equation:  $ax^2 + bx + c = 0$       (last run)
```

```
Please input a > 1
```

```
Please input b > -2
```

```
Please input c > -3
```

```
There are two distinct real roots x1 = 3.000000, x2 = -1.000000
```

4 Logic

Open: `ex_logic/main.c`

Task:

Write a program that decides if a person holding a credit card is allowed to do a certain purchase. At this very hypothetical bank the credit score is simply a factor between 0 and 1, and the rules when doing a purchase are as follows:

- If the account balance is larger than, or equal to, the cost, then all is OK.
- If the account balance is less than the cost:
 - The bank does not want to pay more than 70% of the total cost of a purchase, so the credit factor must be above 0.3.
 - If the account balance is larger than, or equal to, the cost multiplied by the credit factor, then all is OK.

Use logical operators (and, or) to express the above rules.

Example output:

```
Enter account balance: 500
Enter credit factor (0-1): 0.7
Enter cost of purchase: 400
!!!ALL OK!!!

Enter account balance: 200
Enter credit factor (0-1): 0.7
Enter cost of purchase: 300
!!!NOT OK!!!

Enter account balance: 100
Enter credit factor (0-1): 0.5
Enter cost of purchase: 200
!!!ALL OK!!!

Enter account balance: 100
Enter credit factor (0-1): 0.2
Enter cost of purchase: 80
!!!ALL OK!!!
```

5 Coffee price

Open: `ex_coffeeprice/main.c`

Task:

Write a program that calculates the price of a number of coffees. In this situation, a customer will order a number of coffees and then hand over a discount card. The program you will write functions as the cash register, processing the discount and then telling you the final price.

Take note of the following things:

- The discount entered is a percentage, e.g. input 25 is a 25% discount.
- The cash register contains kronor, but no öre. Therefore, you must round the price to the nearest krona.

Example output:

```
Enter the number of coffees: 4
Enter the customer's discount percentage: 16

Total cost before discount: 140.00 kr
Total cost after discount: 118.00 kr
```

6 Circle

Open: `ex_circle/main.c`

Samples: `sample_functions`

This is an exercise where you practice functional decomposition of a program. The given program calculates the area and circumference of a circle and your task is to decompose the program into functions.

Tasks:

- First, run the program as it is to see how it works.
- Decompose the program such that it uses the following functions.
 - `get_radius`: asks the user for the radius of the circle.
 - `circle_circumference`: calculates the circumference of the circle.
 - `circle_area`: calculates the area of the circle.
- Make sure that the program works as it did before the decomposition.

Note: When you use functions from `math.h`, remember to link the math library by adding `-lm` at the end of your compile command. See note on quadratic function exercise.

7 Functions

Open: `ex_functions/main.c`

Samples: `sample_functions`

In this exercise you will implement functions and see how simple "test macros" can be used to check that the functions work as they should.

Tasks:

- Implement all functions that are declared above `main`.
- Implement one function at the time (start with the square function) and uncomment the proper lines in 'main' to test the function before moving on to the next function. That is, the program output should be "ok" for the current test before moving on.
- Make sure that all tests are passed, the program should output 22 rows of "ok" when all functions have been implemented.

Note: When you use functions from `math.h`, remember to link the math library by adding `-lm` at the end of your compile command. See note on quadratic function exercise.

8 Patterns

Open: `ex_patterns/main.c`

Samples: `sample_puzzlers`

Task:

Write a program that prints the patterns shown below. You can decide yourself how many lines to print for each pattern, but at least as many as in the example. You should use: `while` and `printf`.

Example output:

```
X                (first pattern)
XX
XXX
XXXX
XXXXX
XXXXXX
XXXXXXX
...

XXXXXXXXXX      (second pattern)
XXXXXXXXXX
 XXXXXXXXXX
  XXXXXXXXXX
   XXXXXXXXXX
    ...
```

9 Nim

Open: ex_nim/main.c

Samples: sample_functions, sample_nested_stmt

In this exercise you will implement a variant of the game Nim (see <https://en.wikipedia.org/wiki/Nim>). In the game, it is arbitrarily chosen who starts playing the game (human or computer). In this variant, there are 13 coins in a pile, and in each turn one can take a maximum of 3 coins. You lose if you are forced to pick the last coin in the pile.

Tasks:

- Implement the Nim game using functional decomposition. Any non-trivial task should be handled by a function.
- Start by implementing the functions one at the time and make sure that they work as you expect.
- When all functions are working, use them to create one round of the game.
- When one round works, use a loop to run the full game.

Example output:

```
Welcome to NIM
There's 13 coins in the pile
Pick coins > 3
Player took 3
There's 10 coins in the pile
Computer took 2
There's 8 coins in the pile
Pick coins > 0                (force player to take correct)
Pick coins > 4                (force player to take correct)
Pick coins > 3
Player took 3
There's 5 coins in the pile
Computer took 2
There's 3 coins in the pile
Pick coins > 2
Player took 2
There's 1 coins in the pile  (i.e. computer forced to pick last = LOSE)
Game over.
Winner is: Human
```

10 Day number

Open: ex_day_nbr/main.c

Task:

This program calculates the ordinal number of a day given a year and a date. Your task is to decompose the top level function into smaller functions while not changing the output of the program. **NOTE:** you should not change anything in main except uncommenting the test function to run the tests.

The top level function `get_day_nbr` takes the current year, month, and day as input. It should do the following:

- Sum all days in previous months and add the number of days in the current month.
- Add one "leap day" if the current year is a leap year and current month is > 2.

Additional tips:

- Try to sketch a solution before starting to code. Break down the various tasks that `get_day_nbr` should do into smaller tasks that you can implement as separate functions.
- For each smaller function you implement, add a suitable test (using the test macro) in the test function and make sure the function works.
- Reference: <https://www.epochconverter.com/daynumbers>

Example output:

```
Input the year > 2019
Input the month number > 2
Input the day number > 12
Ordinal number for 12/2 in 2019 is: 43
```

11 Sin

Open: `ex_sin/main.c`

This is an old exam question. Try to solve this with pen and paper before using the computer, as this is how you would've had to do it during the exam.

Task:

Funktionen $\sin(x)$ kan uppskattas med denna serieutveckling: $\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots + (-1)^n \frac{x^{(2 \cdot n + 1)}}{(2 \cdot n + 1)!}$ där $(2 \cdot n + 1)!$ är fakultet av $(2 \cdot n + 1)$. Fakultet av n är $n \cdot (n-1) \cdot \dots \cdot 2 \cdot 1$. Ju större x desto fler termer måste inkluderas i serien.

Skriv en funktion som beräknar $\sin(x)$ enligt serien ovan. Serien ska avslutas när absolutbeloppet av termen som läggs till är mindre än 0.001. Full funktionell nedbrytning krävs för full poäng.

I `main` anropa funktionen för `x = 2.0` och skriv ut resultatet. Anropa även `sin` från `math.h` och skriv ut skillnaden mellan funktionerna.

Example output:

```
sin(x) by series: 0.909296
Difference to std. function: -0.000001
```